Thank you for downloading Heart System!

If you have any questions about this asset, then feel free to reach out through email: dreamnoms@gmail.com

Contents

Getting Started

The best place to start is by looking at the Example scene.

The Example scene contains a list of all the functions and properties that you can call on the HealthController or HeartEffectSystem script.

Additionally, the Example scene contains an example script (ExampleEventSubscriber) that prints the HeartEvents that are called in realtime. Check out the Console while interacting with the buttons to see what events are called!

Setting it up in your scene

To add the Heart System to your scene, go to the Prefab folder and drag the "HeartContainer.prefab" onto a Canvas in your scene. The HeartContainer has 3 heart children to help visualize what it will look like when the game starts. You shouldn't make any changes to these children.

If you want to control the number of hearts or the position, then make changes to the HeartContainer. If you want to change the appearance of the hearts then make changes to the "Heart.prefab."

Position and Layout (Heart Container)

Change the anchor, width, and height of its RectTransform to control the overall size and position of the heart container. You can delete the Image component if you don't want a background.

GridLayoutGroup determines the size, spacing, and alignment of the hearts.

Change the Cell Size to change the size of the hearts.

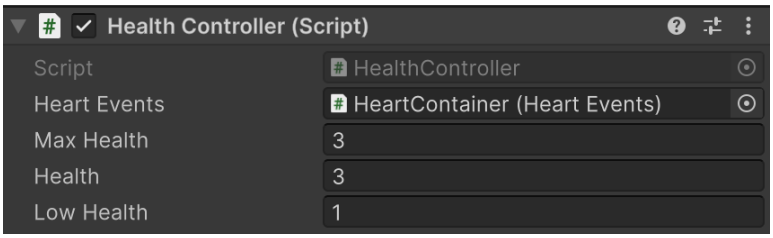
To change the size of the gap around the hearts, change the Spacing (space between hearts) or the Padding (space between hearts and container).

Start Axis determines the direction of the hearts. It can be vertical or horizontal.

Child Alignment determines the position of the hearts in the container.

Changing number of hearts (Heart Container)

Health Controller (Script) on the HeartContainer object is where you can change the health of the player:



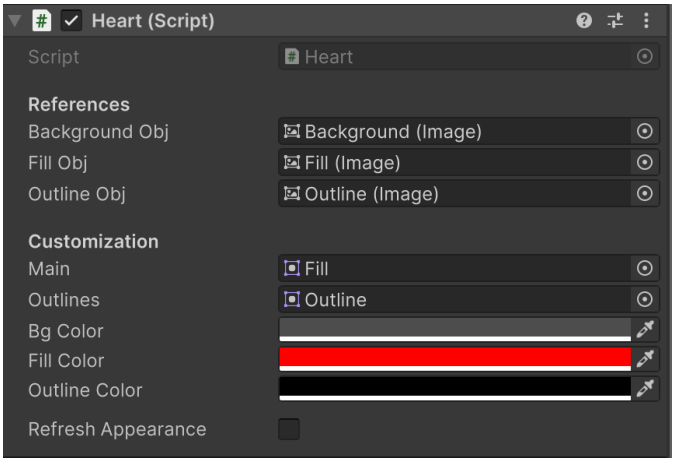
Max Health: determines how many hearts the player has.

Health: The current health of the player. Make it the same value as Max Health if you want the player to begin with full health.

Low Health: When the player gets at this health or lower, this script will trigger a OnEnteredLowHealth event in the Heart Events. The event doesn't do anything on its own, but you can subscribe to it to implement your own logic. For more information on events see the Event section of this README.

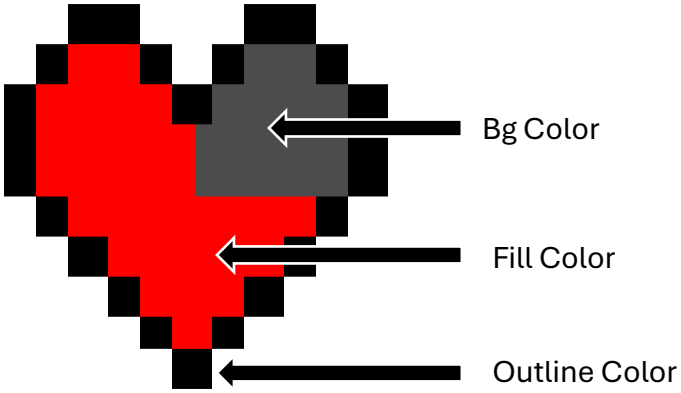
Changing Heart Appearance (Heart Prefab)

The Heart script on the Heart prefab contains several Customization options:



Main: The main sprite used for the Heart. Its color should be white if you want effects to be applied correctly.

Outlines: A sprite image that outlines the Heart. Its color should also be white. If you don't want your heart to have any outlines, then on the Heart prefab delete the "Outline" child.



Bg Color: The color of an empty heart

Fill Color: The main color of the heart

Outline Color: The color of the outlines around the heart.

Clicking the Refresh Appearance will update the heart to match the values in the inspector. The checkbox will remain unchecked, but the appearance of the heart should update.

Effects

Heart Effect System (Script) manages the effects on the hearts. You can apply an effect to a heart by calling the BeginEffect function on that script.

BeginEffect takes in two parameters: a HeartEffect Scriptable object and an optional BeginEffectMode enum value (default is Single).

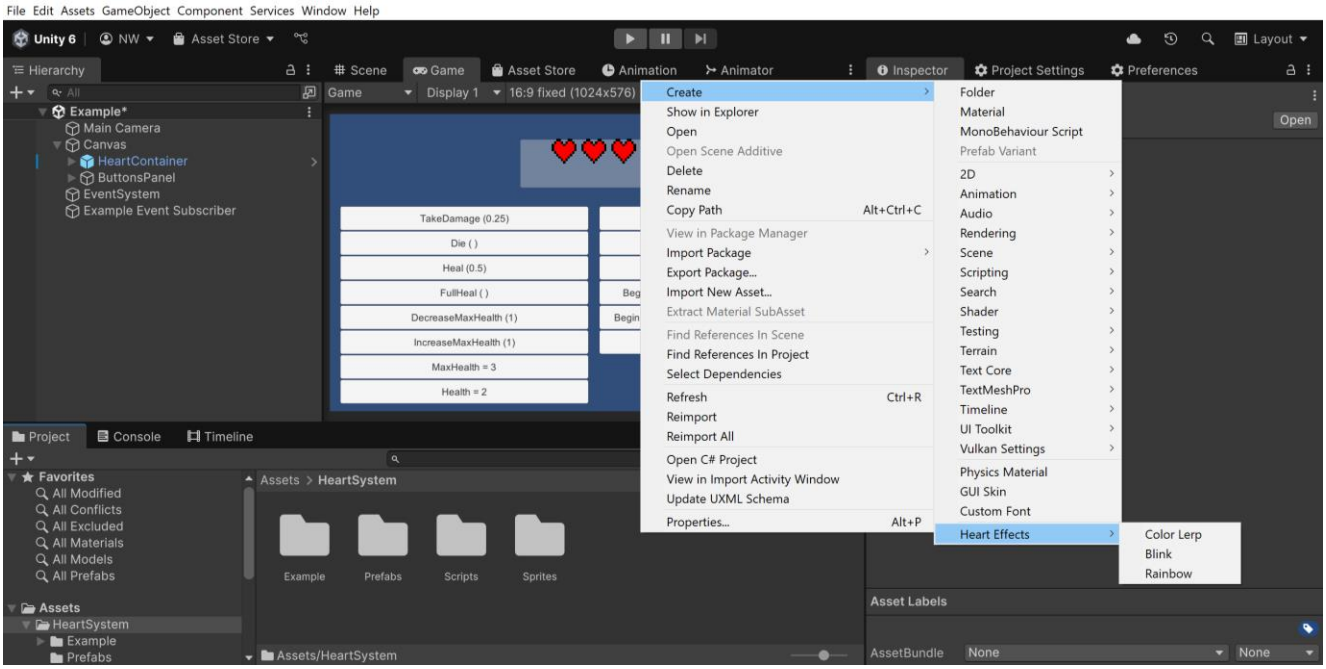
BeginEffectMode.Additive will stack effects on the heart, but it can be unpredictable with some effects. It is recommended to stick to the default of Single to prevent glitches.

To stop a specific effect, call the StopEffect(HeartEffectSO effect) function with the parameter being the effect you want to stop.

To stop all effects, call the StopAllEffects( ) function.

Creating a New Effect

To create a new effect, right click in your project go to Create>Heart Effects and choose one of the 3 effects:



Color Lerp: Allows you to specify a custom gradient that will set the color of the hearts over time. It cannot be stacked with any other effect.

Blink: Hearts will alternate between the main color and a specified blink color. Can be stacked with Rainbow effect.

Rainbow: The hue of the hearts will change over time. It can be stacked with the Blink effect.

Effect Parameters

Repeat Rate: The time in seconds to wait between each call to the effect. The bigger the value, the slower the effect. It is recommended to be a minimum value of 0.1

Smoothness (Color Lerp and Rainbow only): How gradual the color will change.

Wave effect (Color Lerp and Rainbow only): The direction and strength of a wave effect along the chain of hearts. Negative values: colors move right to left. Positive values: colors move left to right. Zero: there is no wave along the chain of hearts; all hearts will be the same color

Events

Heart Events (Script) holds events that you can subscribe to. The events are automatically called by Heart Controller (Script), so **do not** call any of the "Raise" functions in that script.

To subscribe to an event, you need to include the DreamNoms.HeartSystem.EventSystem namespace in your script.

Include a reference to the HeartEvents object. In the inspector set its value to the heart system that you want to listen for.

In OnEnable, subscribe to an event you want to listen for by using the += operator.

In OnDisable, unsubscribe from the events you subscribed to using the -= operator to prevent memory leaks.

Here is a simple script that will call the function MyHealthChanged when the HeartEvent "OnHealthChanged" was raised:

```
using UnityEngine;

// Require the event namespace
using DreamNoms.HeartSystem.EventSystem;

Unity Script (1 asset reference) | 0 references
public class ExampleEventSubscriber : MonoBehaviour
{
    //Include a reference to the heart event you want to listen for
    [SerializeField]
    private HeartEvents heartEvents;

    //Subscribe to the events you want to listen to in OnEnable
    Unity Message | 0 references
    private void OnEnable()
    {
        heartEvents.OnHealthChanged += MyHealthChanged;
    }

    //Unsubscribe from the events you have subscribed to in OnDisable
    Unity Message | 0 references
    private void OnDisable()
    {
        heartEvents.OnHealthChanged -= MyHealthChanged;
    }

    2 references
    public void MyHealthChanged(float oldHealth, float newHealth)
    {
        if (oldHealth > newHealth)
        {
            Debug.Log("Ow! That hurt!");
        }
        else
        {
            Debug.Log("You have been healed by " + (newHealth - oldHealth) + " HP");
        }
    }

}
```

Event List

OnMaxHealthChanged (int oldMaxHealth, int newMaxHealth)

Called when maximum health changes (eg player gained or lost a heart)

OnHealthChanged (float oldHealth, float newHealth)

Called when current health is increased or decreased.

OnDeath ( )

Called when health reaches 0

OnRevive ( )

Called when previously dead, but health increases

OnEnteredLowHealth ( )

Called when health gets below or equal to Low Health value in the Heart Container's Health Controller script.

OnExitLowHealth ( )

Called when health was low, but is no longer low

OnEnteredFullHealth ( )

Called when health reaches full

OnExitFullHealth ( )

Called when it is no longer at full health